Operator, Sequence, Flowchart, Pseudocode

Programming Fundamentals Teaching Team 2026

Objectives

After studying this material, students should be able to:

1. Understand and describe about Operators (Arithmetic Assignment, Combined Assignment, Increment, Decrement, Relational, Logic, Conditional, Bitwise, Casting)
2. Understand and implement sequence in Java Programming
3. Describe and explain flowcharts including the its symbols
4. Understand and be able to create algorithms in the form of flowcharts.
5. Understand and be able to use pseudocode and flowcharts to model the algorithm

OPERATOR

Operator

- Operators are symbols commonly used in writing a statement in any programming language. The operator will perform an operation on the operand according to its function.
- Examples of operations include addition, subtraction, division and so on.

```
3 + 8 * 4
3  8  4  is operand
 +  * is Operator
```

Operator Types

1. Arithmetic Operators
2. Increment and Decrement Operators
3. Assignment Operators
4. Relational Operators
5. Logical Operators
6. Bitwise Operators

1. Arithmetic Operators

Arithmetic operator is an operator that functions for arithmetic operations.

Operator	Meaning	Example	Result
+	Addition	10 + 2	12
-	Subtraction	10 – 2	8
*	Multiplication	10 * 2	20
/	Division	10 / 2	5
%	Modulus (remainder)	10 % 2	0

2. Increment and Decrement Operators

The Increment and Decrement operators are used to increase or decrease an integer value by one unit and can only be used on variables.

Name	Operator	Meaning
Pre increment operator	++x	Add 1 to x, then use new value of x
Post increment operator	x++	Use value of x, then add 1 to x
Pre decrement operator	--x	Take 1 from x, then use new value of x
Post decrement operator	x--	Use value of x, then take 1 from x

```java
public class OperatorIncrementdanDecrement {
    public static void main(String[] args) {
        int i = 1;
    //increment
        System.out.println("i : " + i);
        System.out.println("++i : " + ++i);
        System.out.println("i++ : " + i++);
    //decrement
        System.out.println("--i : " + --i);
        System.out.println("i--: " + i--);
        System.out.println("i : " + i);
```

variabeltipedataoperator.OperatorIncrementdanDecrement > main

out - variabeltipedataoperator (run)

```
run:
i : 1
++i : 2
i++ : 2
--i : 2
i--: 2
i : 1
```

Example

```
int x = 5;
int y = ++x; //x is increased into 6, then y = 6

//result:
// x = 6
// y = 6
```

```
int x = 5;
int y = x++; //y got the first value of x (5), then x increased into 6

//result:
// x = 6
// y = 5
```

- ++x means "increase x first, then use the result."
- How the execution goes:

1. X is increased from 5 to 6
2. New value of x (which is 6) is stored on y

- X++ means "use the value of x, then increase the value"
- How the execution goes:

1. The current value of x (which is 5) is assigned to y.
2. Then x is incremented from 5 to 6.

2. Assignment Operators

Java assignment operators are used to assign a value to a variable. The assignment operator is simply '=',

Operator	Usage	Equivalent To
+=	Op1 += Op2	Op1 = Op1 + Op2
-=	Op1 -= Op2	Op1 = Op1 - Op2
*=	Op1 *= Op2	Op1 = Op1 * Op2
/=	Op1 /= Op2	Op1 = Op1 / Op2
%=	Op1 %= Op2	Op1 = Op1 % Op2
&=	Op1 &= Op2	Op1 = Op1 & Op2
|=	Op1 |= Op2	Op1 = Op1 | Op2
^=	Op1 ^= Op2	Op1 = Op1 ^ Op2

2. Assignment Operators (continued)

- a = a + 5; can be shortened to a + = 5;
- b = b – 5; can be shortened to b - = 1;
- c = c * 5; can be shortened to c * = 3;
- d = d / 5; can be shortened to d / = 5;
- e = e% 5; can be shortened to e% = 5;

```java
public class operatorassigment2 {
    public static void main(String[] args) {
    int a = 10;
    // Demo operator assignment
    a += 5;
    System.out.println("value a [10] += 5 = " + a);

    int b = 10;
    b -= 5;
    System.out.println("value b [10] -= 5 = " + b);

    int c = 10;
    c *= 5;
    System.out.println("value c [10] *= 5 = " + c);

    int d = 10;
    d /= 5;
    System.out.println("value d [10] /= 5 = " + d);

    int e = 10;
    e %= 5;
    System.out.println("value e [10] %= 5 = " + e);
  }
```

Output - variabeltipedataoperator (run)

```
run:
value a [10] += 5 = 15
value b [10] -= 5 = 5
value c [10] *= 5 = 50
value d [10] /= 5 = 2
value e [10] %= 5 = 0
BUILD SUCCESSFUL (total time: 0 seconds)
```

4. Relational Operators

Relational operators in Java are used to generate boolean values which are often used to control the flow of a program.

Operator	Meaning	Example	Result
<	Less than	5 < 2	False
>	Greater than	7 > 2	True
<=	Less than or equal to	9 <= 9	True
>=	Greater than or equal to	4 >= 1	True
==	Equal to	6 == 3	False
!=	Not equal to	6 != 8	True

```java
public class relational {
    public static void main(String[] args) {
        int x, y, z;
        x = 100;
        y = 99;
        z = 99;
        System.out.println("The value of x is = " + x);
        System.out.println("The value of y is = " + y);
        System.out.println("The value of z is = " + z);
        System.out.println("The result of y == z is " + (y == z));
        System.out.println("The result of y < z is " + (y < z));
        System.out.println("The result of x != y is " + (x != y));
        System.out.println("The result of x >= z is " + (x >= z));
    }
}
```

```
run:
The value of x is = 100
The value of y is = 99
The value of z is = 99
The result of y == z is true
The result of y < z is false
The result of x != y is true
The result of x >= z is true
BUILD SUCCESSFUL (total time: 0 seconds)
```

5. Logical Operators

This operator is used for logical expressions that return boolean values. The operators used are AND (&&), OR (||) and NOT (!).

Operator	Meaning	Example	Result
&&	AND	x = 6 y = 3 (x < 10) && (y > 1)	True
||	OR	x = 6 y = 3 (x == 5) || (y == 9)	False
!	NOT	x = 6 y = 3 !(x == y)	True

```java
public class operatorlogika {
    public static void main(String[] args) {
        boolean _true = true;
        boolean _false = false;

        System.out.println("Relation with OR (||)");
        System.out.println("_true || _true : " +(_true||_true));
        System.out.println("_true || _false : " +(_true||_false));
        System.out.println("_false || _true : " +(_false||_true));
        System.out.println("_false || _false : " +(_false||_false));

        System.out.println("Relation with AND (&&)");
        System.out.println("_true && _true : " +(_true&&_true));
        System.out.println("_true && _false : " +(_true&&_false));
        System.out.println("_false && _true : " +(_false&&_true));
        System.out.println("_false && _false : " +(_false&&_false));
        System.out.println("Relation with NOT (!)");
        System.out.println("inverse of (NOT) _true is: " +!_true);
        System.out.println("inverse of (NOT) _false is: " +!_false);
    }
```

```
run:
Relation with OR (||)
_true || _true : true
_true || _false : true
_false || _true : true
_false || _false : false
Relation with AND (&&)
_true && _true : true
_true && _false : false
_false && _true : false
_false && _false : false
Relation with NOT (!)
inverse of (NOT) _true is: false
inverse of (NOT) _false is: true
BUILD SUCCESSFUL (total time: 1 second)
```

6. Bitwise Operators

- This operator is used to perform bit manipulation of a number
- Bitwise operator types:
  a. Bitwise OR (|)
  b. Bitwise AND (&)
  c. Bitwise XOR (^)
  d. Bitwise Complement (~)

6. Bitwise Operators

- Bitwise OR (|)

  The result of the bit is 1 when one of the bits is 1, otherwise it is 0.
- Example:

```
int a = 5;//0101
int b = 7;//0111
System.out.println(a|b);//output 7
//0101
//0111
//____
//0111 -> 7
```

6. Bitwise Operators

- Bitwise AND (&)

  The result of the bit is 1 when all the bits are 1, otherwise it is 0.
- Example:

```
int a = 5;//0101
int b = 7;//0111
System.out.println(a&b);//output 5
//0101
//0111
//____
//0101 -> 5
```

6. Bitwise Operators

- Bitwise XOR (^)

  Bit value is 1 when there are bits of 1 and 0, otherwise it is 0.
- Example:

```
int a = 5;//0101
int b = 7;//0111
System.out.println(a^b);//output 2
//0101
//0111
//____
//0010 -> 2
```

6. Bitwise Operators

- Bitwise Complement (~)

  The inverse bit value, when the bit value is 1 it produces 0 while the value 0 produces 1.

- Example:

```
int a = 5;//0101
System.out.println(~a);//output -6
//0101
//____
//1010 -> 10
```

~n = -(n+1)
~(-n) = (n-1)

SEQUENCE

Sequence…(1)

- Sequence is **the flow of instruction execution**, where each instruction is executed **sequentially** according to the order in which it is written in the program code.
- A program will follow **the instruction order** specified by the programmer. For example, if there are three instructions, A, B, and C, then A will be executed first, followed by B, and then C.
- This execution order **is crucial** to ensuring the program functions according to its intended logic. Incorrect instruction order can result in incorrect output or undesired program behavior.

Sequence…(2)

Please Pay attention to the following code!

```
int a = 10;
int b = a + 5;
```

1. First line: Initializes variable a with the value 10.
2. Second line: Initializes variable b with the result of a + 5 (i.e., 10 + 5 = 15).

This is a simple example of how instructions are executed in a predetermined order to achieve the correct result.

Sequence…(3)

Towers of Hanoi (Example for 3 Discs)

A pylon (A) contains three discs (red, blue, and green) with red at the bottom, then blue, and green at the top. Your goal is to move the entire stack of discs from pylon A to pylon C, using pylon B as a support, while following these rules:

1. **One Disc per Move**: Only one disc may be moved at a time.
2. **Larger Disc on the Bottom**: A larger disc may not be placed on top of a smaller disc.

Sequence…(4)

Solution Steps (for 3 Discs)

1. Move the smallest disc: (green) from A to C.
2. Move the middle disc: (blue) from A to B.
3. Move the green disc: from C to B.
4. Move the largest disc: (red) from A to C.
5. Move the green disc: from B to A.
6. Move the blue disc: from B to C.
7. Move the green disc: from A to C.

Result: After 7 steps, the stack of discs has been successfully moved from pole A to pole C, with each disc in its proper position.

PSEUDOCODE

Pseudocode

- Pseudocode is a high-level description of a computer programming algorithm that uses **notation** that **corresponds** to **programming code**. The **notation of pseudocode** is **not related** to a **specific programming language**
- Pseudocode is used to design and to model algorithm (the steps to solve a logical problem). It could explain the algorithm clearly and easily understood by humans, not by machines.
- Written in a simple English or Indonesian
- **Syntax** or **notation** is used to describe **commands** (for example: “if”, “repeat”, “until”, “if”, “repeat”, “until”)
- It is called “pseudo” because it is not a real programming language, but rather an intermediary form between human language and program code.

Pseudocode

The following syntaxes are the most basic notation in pseudocode:

1. Input the data (**Input**)
2. Output the data(**Output**)
3. Calculate or operate the data(**Compute**)
4. Save the data (**Store**)
5. Compare the data (**Compare**)
6. Relational Operations
7. Selection (**Select)**
8. Repeat process (**Loop**)

1. Input

- The syntaxes that could be used to read input or information in pseudocode are **“Read”, “Get”, “Baca” , “Input”**
- Example:

  Read number

  Get taxCode

  Baca name

  Input phonenumber1

2. Output

- The following syntaxes are notation to display output or information, the statements that commonly used are **“Print”, “Write”, “Put”, “Output”, “Display”** or **“Cetak”**
- Example:

  Print “Politeknik Negeri Malang”

  Cetak “Arithmetic Operation”

  Output Total

  Display hasil

3. Computation

- To do some computation, the symbol of operator could be utilized. For example, to perform an arithmetic operation we can use:

+	add
-	subtract
*	multiply
/	divide
()	bracket

- Statement **“Compute”, “Calculate”** or **“Hitung”** could be used as well.
- Example:

```
Compute  X = (1+2) * 3
Total = Total + 10
```

4. Store the Data

- There are 3 options to store the data:
  - Giving **initial value → "Initialize"** or **"Set"**
  - Giving a value as **the result of operation → "="**
  - Giving a **specific value** for a **variable → "Save"** or **"Store"**
- Example:

  Set age **to** 17
  Initialize counter to 0
  harga = hargaSatuan * jumlah
  Store 50 **to** Discount
  total ← number1 + number 2

5. Comparison

- One of the most fundamentals process to **compare the data**, and based on the **comparison result** (true/false) the program will **select the further process**
- To compare the values between two variables, use

<	Less than
<=	Less than or equal to
>	More than
>=	More than or equal to
==	Equal to
!=	Not equal to

- This operation produces a true or false condition.

  Example:

  a > b

  c == d

6. Relational Operations

- Usually used to relate two states or conditions.
- Produces true and false values.
- Keywords: AND, OR.
- Example:

  age < 15 AND value > 70.

  value == 'A' OR value == 'B'

7. Selection

- One of the most fundamentals process to **compare the data**, and based on the **comparison result** (true/false) the program will **select the further process**
- The syntaxes that can be used**: "IF", "THEN"** dan **"ELSE"**
- Example

```
INPUT harga
IF harga >100 THEN
   harga = harga-(0.5*harga)
ELSE
   harga = harga-(0.1*harga)
ENDIF
OUTPUT harga
```

8. Looping

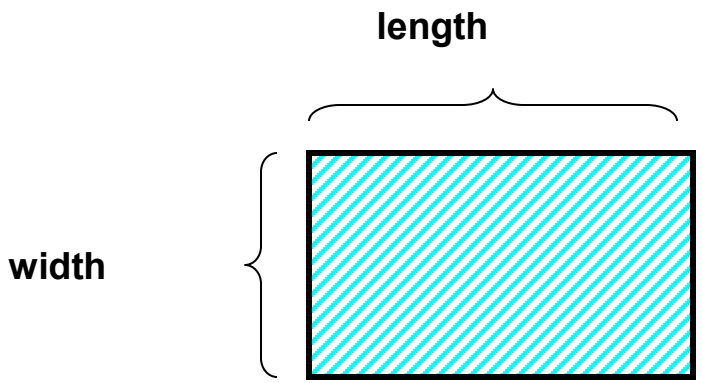
- If there is any process that needs to be run many times in a specific pattern, until reaching a specific conditions, we can us syntax → **“DOWHILE”** and **“ENDDO”**.
- Example

```
number = 0
DOWHILE number < 10
    output number
     number = number +1
ENDDO
```

Example

Create an algorithm using pseudocode to calculate the area of a rectangle, with the following formula:

area = length * width



Pseudocode Solution

Algorithm: rectangle area calculation

{input length and width and then calculate the area of rectangle}

Declaration :

length, width, area: **int**

Description :

1. **print** “Program to Calculate Area of a Rectangle”
2. **print** “Input length = ”
3. **read** length;
4. **print** “Input width= ”
5. **read** width
6. area = length * width
7. **print** “The area is = ”
8. **print** area

FLOWCHART

Flowchart

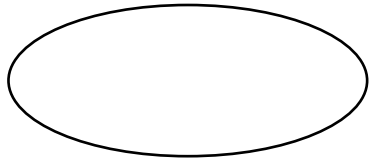
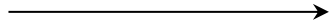
- A flowchart is a **diagram** with specific symbols used to **explain the sequence of processes** and **its connectivity** to **solve the problem**
- One of algorithm modelling.
- There are 2 types of flowchart:
  1. System Flowchart → steps of process in the system, that shows input, output, storage in correlation with data processing
  2. Program Flowchart → steps of instructions that is illustrated by using a graphical notation, to model the algorithm of a program
- Draw.io is a software tool for creating flowcharts.

Flowchart

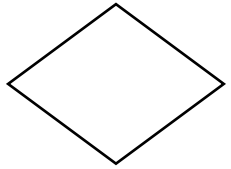
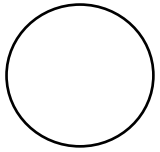
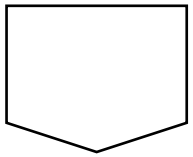
- If Pseudocode models the algorithm using a text-based description, a Flowchart models the algorithm using a graphical notation instead.
- Flowchart uses **graphical symbols** to **illustrate the steps of the processe**s and its **connectivity** to **solve the problem**
- One of algorithm modelling.
- There are 2 types of flowchart:

1. System Flowchart → steps of process in the system, that shows input, output, storage in correlation with data processing
2. Program Flowchart → steps of instructions that is illustrated by using a graphical notation, to model the algorithm of a program

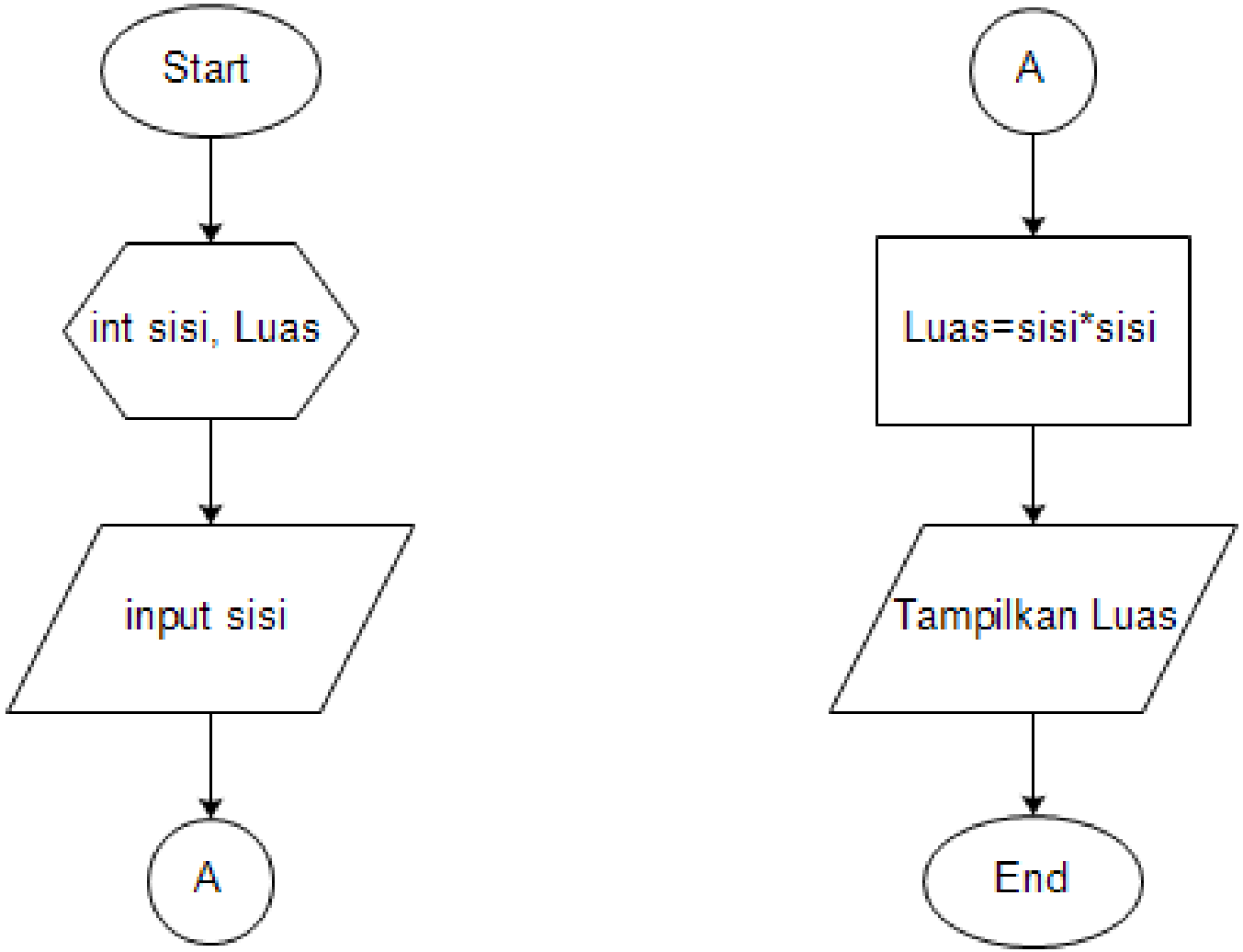
Flowchart Symbols

Symbol	Name	Description
	Terminator	Symbol for the beginning (start) and finish (end) of an algorithm
	Preparation	Symbol for preparing storage that will be used
	Input-output	Simbol for input in output
	Flow Line	Symbol that illustrates the flow of process

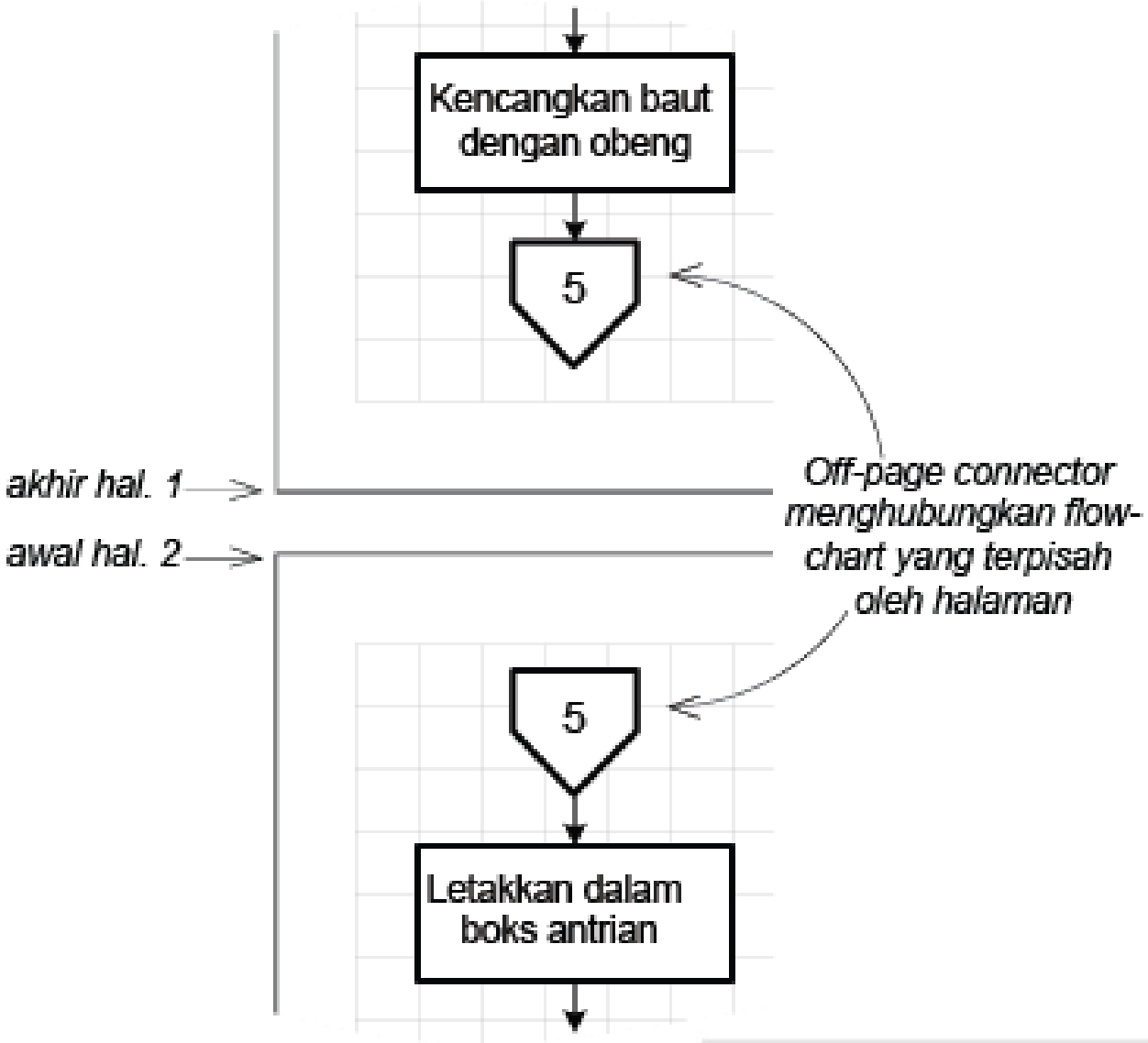
Flowchart Symbols

Symbol	Name	Description
	Process	Symbol to illustrate the process
	Branching/Condition Selection	To check or select the condition (it results true or false)
	On Page Connector	Connecting the flowchart in the same page
	Off Page Connector	Connecting the flowchart in the different page
	Predefined Process	Symbol that illustrates the sub-process or sub-program, function, procedure

Example for On Page Connector

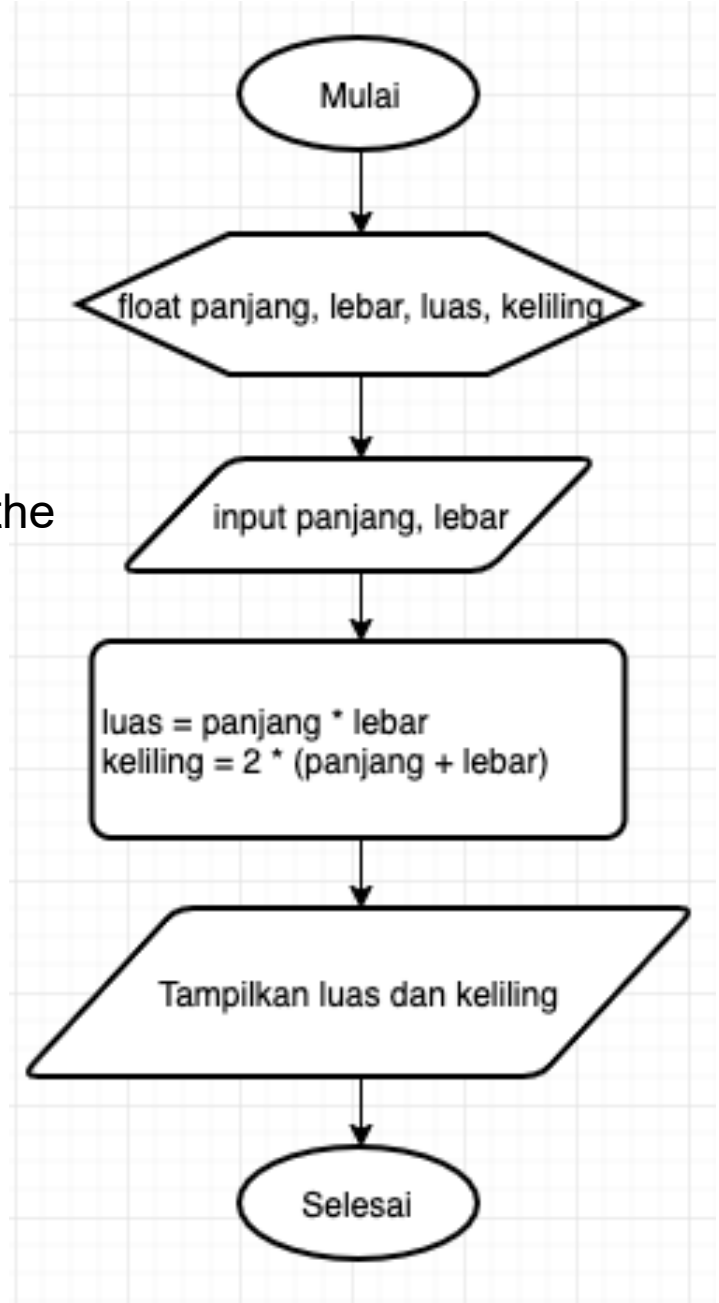


Example for Off Page Connector



Flowchart Example

Pak Adi has a rectangular garden, make a flowchart to calculate the area and perimeter of Pak Adi's garden!



Example (Based on Week 2)

Pak Adi has a rectangular garden. Pak Adi wants to make a wooden fence to surround the garden. Before creating a program to help Pak Adi to calculate the perimeter of his garden, help Pak Adi to identify variables and data types along with the algorithm!

Example

1. Algorithm

Input: length, width

Output: Perimeter

Process:

1. input length, width
2. Perimeter = 2 x (length + width)
3. Output perimeter

2. Variable Identification

Variable	Data type
length	int
width	int
perimeter	int

PSEUDOCODE

Algorithm : Perimeter Calculation

{Input length and width of the rectangle and calculate the perimeter}!

Declaration :

length, width, perimeter: int

Description :

1. **read** length
2. **read** width
3. perimeter = length * width
4. **print** perimeter

Example

PSEUDOCODE

Algorithm : perimeter Calculation

{Input length and width of the rectangle and calculate the perimeter}!

Declaration :

length, width, perimeter : in

Line 8,9,10

Description :

1. **read** length
2. **read** width

Line 12,13

3. perimeter = length * width

Line 15

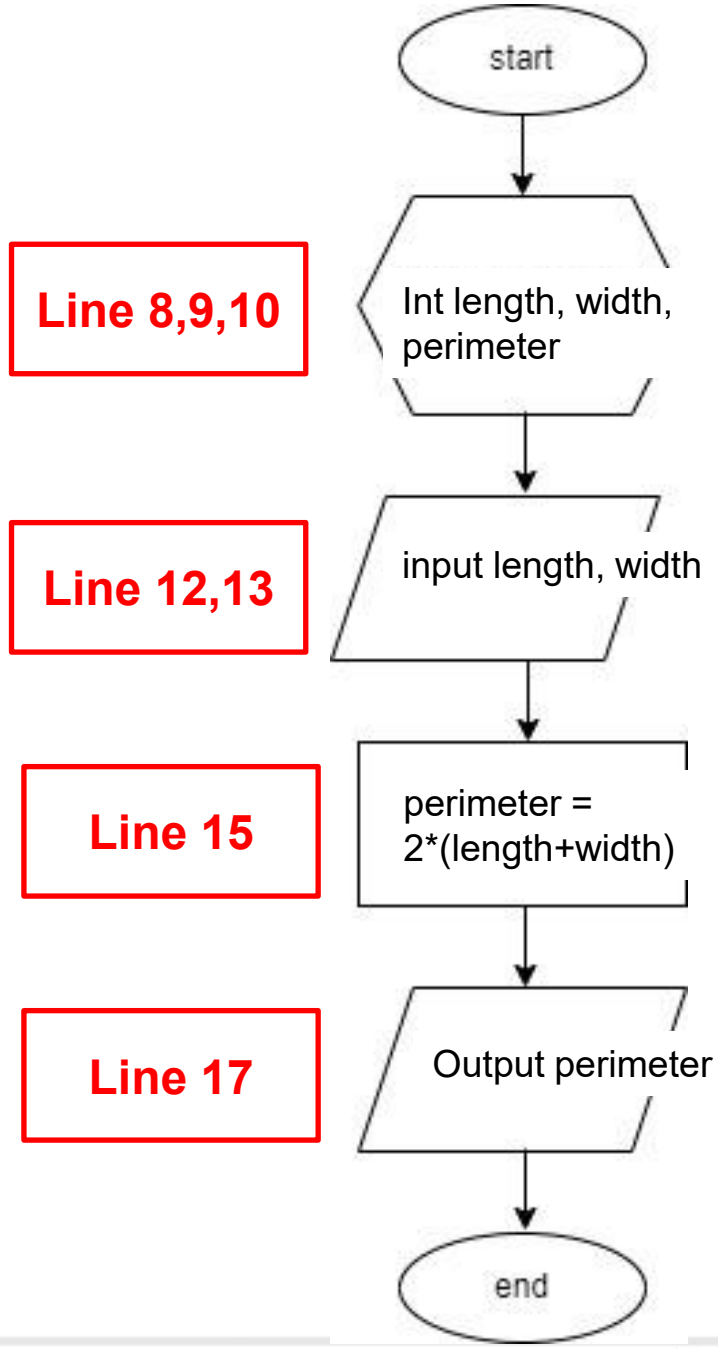
4. **print** perimeter

Line 17

```
4   public class Rectangle {
        Run | Debug
5       public static void main(String[] args) {
6           Scanner input = new Scanner(System.in);
7
8           int length;
9           int width;
10          int perimeter;
11
12          length = input.nextInt();
13          width = input.nextInt();
14
15          perimeter = 2 * (length + width);
16
17          System.out.println(perimeter);
18      }
19  }
20
```

Example

```
4   public class Rectangle {
        Run | Debug
5       public static void main(String[] args) {
6           Scanner input = new Scanner(System.in);
7
8           int length;
9           int width;
10          int perimeter;
11
12          length = input.nextInt();
13          width = input.nextInt();
14
15          perimeter = 2 * (length + width);
16
17          System.out.println(perimeter);
18      }
19  }
20
```



Line 8,9,10

Line 12,13

Line 15

Line 17

Task 1

Mr. Ali purchased a motorcycle for Rp. x on credit. He paid a down payment of Rp. y and the remaining balance was paid in installments over z months with a fixed interest rate of 1% per month on the remaining unpaid price. How many monthly installments does Mr. Ali have to pay?

a. Create a flowchart algorithm for the case study!

b. Create a pseudocode algorithm for the case study!

Task 2

A car travels x km from Malang to Surabaya. The car uses an average of 1 liter of gasoline for 2 km. If the price of gasoline is Rp. 10,000 per liter, how much gasoline will be needed for the trip?

a. Create a flowchart algorithm for the case study!

b. Create a pseudocode algorithm for the case study!